

Università degli Studi di Torino

Dipartimento di informatica



**UNIVERSITÀ
DI TORINO**

Tesi di Laurea Triennale in Informatica

TITOLO

Relatore:

Prof. Matteo Sereno

Correlatore:

Dott. Daniele Baccega

Candidato:

Matteo Sardi

ANNO ACCADEMICO 2025/2026

A very cool quote!
- Author of the quote

Abstract

Titolo: Generazione Sintetica di Tracciati Elettrocardiografici: Una pipeline per l'Allenamento di Modelli di Digitalizzazione Documentale tramite Simulazione di Artefatti Reali.

Contesto e Motivazione: Nonostante la progressiva digitalizzazione dei flussi di lavoro clinici, una vasta quantità di dati storici e diagnostici relativi all'elettrocardiografia (ECG) rimane conservata esclusivamente in formato cartaceo o scansioni analogiche. La conversione automatica di questi documenti in segnali digitali numerici è essenziale per l'analisi computazionale su larga scala. Tuttavia, lo sviluppo di modelli di Computer Vision robusti è spesso ostacolato dalla carenza di dataset annotati che presentino la variabilità e le imperfezioni tipiche dei documenti ospedalieri reali.

Obiettivo: Il presente lavoro propone lo sviluppo di un sistema di generazione sintetica di immagini ECG, finalizzato alla creazione di dataset massivi per l'addestramento di modelli di digitalizzazione. L'obiettivo primario è colmare il divario tra i dati sintetici ideali e la complessità dei tracciati fisici reperibili negli archivi clinici.

Metodologia: Il framework integra tecniche di modellazione del segnale cardiaco con algoritmi di rendering grafico avanzato, producendo in modo procedurale coppie di dati composte da un tracciato "pulito" (ground truth) e la corrispondente istanza degradata. La pipeline modulare include:

1. **Generazione di layout ed intestazioni:** sintesi procedurale di dati anagrafici, formati di reportistica multimodale e griglie millimetriche.
2. **Simulazione di artefatti geometrici:** riproduzione di pieghe, rotazioni, deformazioni prospettiche e variazioni di orientamento del foglio.
3. **Degradazione dell'immagine:** simulazione di artefatti tipici dei processi di scansione, quali variazioni di contrasto, compressione dell'immagine, ombre e rumore del sensore.

Risultati e Conclusioni: L'approccio permette di generare una vasta varietà di campioni con una *ground truth* intrinseca. L'integrazione di artefatti realistici nel processo di generazione sintetica, come suggerito da Li et al. [1] per la robustezza dei modelli in presenza di disturbi strutturali e casuali, risulta fondamentale per superare le criticità legate alla variabilità dei documenti clinici reali. Il sistema si configura come uno strumento essenziale per la valorizzazione del patrimonio informativo contenuto negli archivi cardiologici, ponendo le basi per una completa automazione e digitalizzazione del recupero dati clinici.

Dichiaro di essere responsabile del contenuto dell'elaborato che presento al fine del conseguimento del titolo, di non avere plagiato in tutto o in parte il lavoro prodotto da altri e di aver citato le fonti originali in modo congruente alle normative vigenti in materia di plagio e di diritto d'autore. Sono inoltre consapevole che nel caso la mia dichiarazione risultasse mendace, potrei incorrere nelle sanzioni previste dalla legge e la mia ammissione alla prova finale potrebbe essere negata.

Contents

1	Introduzione	5
1.1	Inquadramento del problema	5
1.2	Obiettivo del lavoro: Pipeline di Rendering e Augmentation	5
1.2.1	Integrazione di Artefatti per la Robustezza	5
1.3	Struttura della tesi	6
1.4	Processo di Trasformazione Immagine	6
1.4.1	Logica di Rendering	7
2	Architettura del Software e Logica di Rendering	9
2.1	Struttura Modulare del Progetto	9
2.2	Formato dei Dati in Ingresso: lo Standard WFDB	10
2.3	Pipeline di Elaborazione Batch	10
2.4	Segmentazione del Segnale e Gestione dei Layout	11
2.4.1	Layout Supportati	11
2.4.2	Segmentazione Temporale per il Formato 3x4	11
2.5	Il Motore di Rendering: <code>ecg_plot.py</code>	12
2.5.1	Calibrazione Fisica della Griglia	12
2.5.2	Configurazione del Canvas Matplotlib	13
2.5.3	Posizionamento delle Derivazioni	13
2.5.4	Colori della Griglia e Stile	13
2.5.5	Impulso di Calibrazione	14
2.5.6	Augmentation Geometrica in Fase di Rendering	14
2.5.7	Annotazioni e Bounding Box	14
2.5.8	Generazione dell'Immagine Pulita	15
2.6	Output del Sistema e Configurazione JSON	15
2.7	Sistema di Configurazione	15

Chapter 1

Introduzione

L'intelligenza artificiale ha aperto nuove frontiere nella gestione dei segnali biomedici. Sistemi basati su architetture di segmentazione come la U-Net sono oggi in grado di riconoscere tracciati ECG su carta e riconvertirli in segnali numerici. Tuttavia, per essere efficaci, questi modelli richiedono una fase di addestramento basata su una vasta quantità di immagini che simulino le imperfezioni del mondo reale.

1.1 Inquadramento del problema

La digitalizzazione automatica degli ECG richiede un processo di ricostruzione del segnale capace di isolare l'attività elettrica del cuore da elementi di disturbo quali griglie millimetriche, pieghe della carta e macchie di inchiostro. La capacità di generalizzazione dei modelli di Deep Learning dipende interamente dalla varietà dei dati su cui vengono allenati; senza un dataset che includa campioni **difficili** e deteriorati, le performance su documenti clinici reali rimangono limitate.

1.2 Obiettivo del lavoro: Pipeline di Rendering e Augmentation

Il presente lavoro si focalizza sulla progettazione di un sistema per la **generazione di immagini sintetiche e l'applicazione di artefatti realistici**. Partendo da un dataset di segnali ECG digitali preesistente, il framework trasforma tali dati numerici in rappresentazioni visuali, sottoponendole a una pipeline di degradazione controllata.

1.2.1 Integrazione di Artefatti per la Robustezza

Il contributo principale risiede nella creazione di un sistema di *data augmentation* avanzato. Come dimostrato da Li et al. [1], l'esposizione del modello a disturbi strutturali durante il training è fondamentale per superare il *domain gap* tra i dati sintetici e le scansioni reali, permettendo di incrementarne la robustezza.

1.3 Struttura della tesi

L'elaborato descrive lo sviluppo e la logica del sistema di trasformazione:

- **Capitolo 2:** Descrizione dell'architettura del software e logica di rendering.
- **Capitolo 3:** Analisi della pipeline di artefatti e gestione della varietà del dataset prodotto.
- **Capitolo 4:** Risultati ottenuti e configurazione sperimentale.

1.4 Processo di Trasformazione Immagine

La pipeline si articola in due stadi principali: la conversione del segnale in immagine e l'iniezione controllata di artefatti. Le **figure 1.1, 1.2 e 1.3** mostrano visivamente il processo: partendo dal segnale digitale grezzo, il modulo di rendering produce una rappresentazione grafica, sulla quale vengono applicati i disturbi per simulare le condizioni di conservazione e scansione dei documenti clinici.

```
89ff c9ff 4000 5600
a5ff 0400 bbff e1ff
0000 e6ff d9ff b1ff
8cff cdff 4100 5300
a6ff 0600 c0ff dcff
fdff e1ff deff b6ff
88ff d4ff 4c00 5200
9eff 1000 c6ff deff
f6ff e4ff e3ff bbff
8bff daff 5000 4d00
...
```

Figure 1.1: Dato grezzo: snippet esadecimale estratto dal dataset ECG.

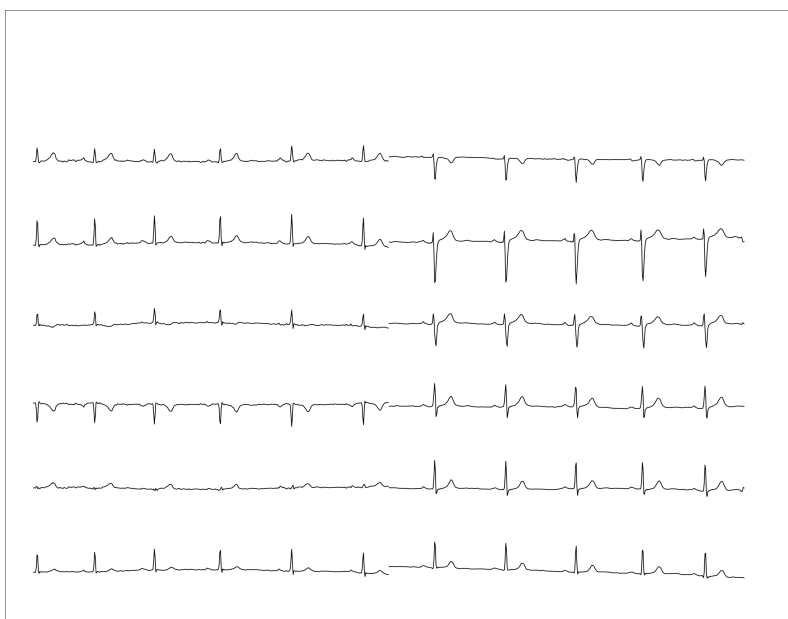


Figure 1.2: Visualizzazione su canvas del segnale ECG: tracciato vettoriale pulito.

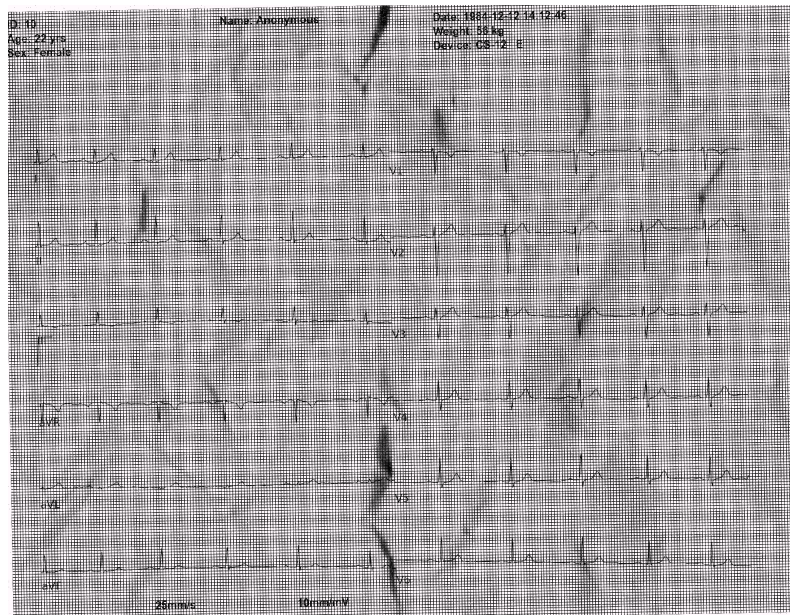


Figure 1.3: Output finale: applicazione di rumore, distorsioni geometriche e altri artefatti per la simulazione di documenti storici.

1.4.1 Logica di Rendering

La trasformazione dal segnale digitale alla rappresentazione grafica è gestita da un motore di generazione basato su librerie di visualizzazione scientifica, configurato specificamente per garantire una corrispondenza millimetrica tra il segnale originale e i pixel del canvas. Tale precisione è fondamentale affinché la risoluzione spaziale risulti conforme agli standard clinici di riferimento. Il sistema opera con una risoluzione di rendering predefinita, ma permette una parametrizzazione flessibile dei parametri grafici per adattare l'output a diverse esigenze di analisi o di simulazione.

Nel prossimo capitolo, verrà analizzata l'architettura del sistema, descrivendo nel dettaglio le fasi di acquisizione dei dati, la configurazione del motore di rendering e l'implementazione tecnica della pipeline di degradazione.

Chapter 2

Architettura del Software e Logica di Rendering

Il presente capitolo illustra la struttura interna del sistema *ecg-image-generator*, descrivendo come i moduli che lo compongono cooperino per trasformare un segnale ECG digitale in un'immagine sintetica pronta per l'uso in pipeline di Deep Learning. Il sistema è stato sviluppato a partire dalla repository pubblica *ECG-Image-Kit* [?], sulla quale sono state apportate significative estensioni e personalizzazioni al fine di aumentarne la flessibilità, le prestazioni e la varietà dell'output prodotto.

2.1 Struttura Modulare del Progetto

Il software è organizzato secondo una architettura a strati ben separati, in cui ogni modulo è responsabile di una fase specifica della pipeline. La struttura delle directory principali è la seguente:

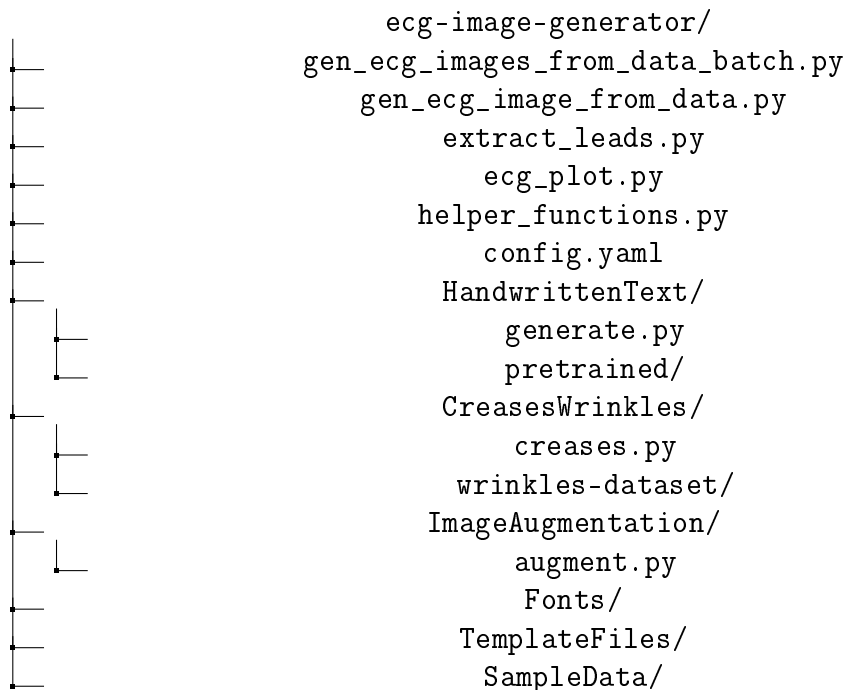


Figure 2.1: Struttura delle directory del modulo *ecg-image-generator*.

I due script principali (`gen_ecg_images_from_data_batch.py` e `gen_ecg_image_from_data.py`) costituiscono i punti di ingresso della pipeline, rispettivamente per l'elaborazione in modalità batch e per la generazione da un singolo file. Il nucleo del rendering è contenuto in `ecg_plot.py`, mentre la logica di estrazione e segmentazione del segnale risiede in `extract_leads.py`. I sottomoduli `HandwrittenText`, `CreasesWrinkles` e `ImageAugmentation` sono responsabili delle fasi di degradazione dell'immagine, descritte in dettaglio nel Capitolo 3.

2.2 Formato dei Dati in Ingresso: lo Standard WFDB

Il sistema accetta segnali ECG nel formato *WFDB* (WaveForm DataBase), definito da PhysioNet [?], che è lo standard de facto per i dataset di segnali biomedici aperti. Ogni registrazione è composta da due file distinti:

- Un file **header** (`.hea`), in formato testuale, contenente i metadati della registrazione: numero di derivazioni, frequenza di campionamento, guadagni del convertitore analogico-digitale (ADC), nomi delle derivazioni, data e ora di acquisizione.
- Un file **dati** (`.dat` oppure `.mat`), contenente i campioni del segnale in formato binario o come matrice numerica.

Il modulo `helper_functions.py` espone le funzioni di basso livello per il parsing di questi file. La funzione `load_header()` legge il file `.hea` come stringa, da cui vengono estratti la frequenza di campionamento tramite `get_frequency()`, i nomi delle derivazioni tramite `get_leads()` e i guadagni ADC tramite `get_adc_gains()`. Il segnale grezzo viene poi caricato da `load_recording()`, che utilizza la libreria `wfdb` per i file `.dat` e `scipy.io` per i file `.mat`, restituendo in entrambi i casi una matrice NumPy di forma (`campioni, derivazioni`).

Un passaggio critico è la normalizzazione dei nomi delle derivazioni, gestita da `standardize_leads()`: la funzione uniforma le convenzioni di notazione, convertendo in maiuscolo le derivazioni standard (es. `i` → `I`) e applicando la notazione camelCase per le derivazioni aumentate (`avr` → `aVR`, `avl` → `aVL`, `avf` → `aVF`).

2.3 Pipeline di Elaborazione Batch

L'elaborazione di un intero dataset è gestita da `gen_ecg_images_from_data_batch.py`. Il flusso è il seguente:

1. **Scansione della directory di input:** la funzione `find_records()` percorre ricorsivamente la directory di input cercando file `.dat` o `.mat`, accoppiando ciascun file dati con il corrispondente header `.hea`. La struttura di sottodirectory viene replicata nella cartella di output.
2. **Preparazione dei task:** per ogni coppia di file viene creata una copia degli argomenti di configurazione, con i percorsi di input e output specifici. Il layout di colonne viene determinato automaticamente in base al parametro `-layout` scelto.

3. **Esecuzione parallela:** i task vengono distribuiti su un pool di processi tramite `ProcessPoolExecutor`, che sfrutta la parallelizzazione a livello di processo per aggirare il Global Interpreter Lock di Python. Questo meccanismo, introdotto come contributo originale al progetto, ha ridotto sensibilmente i tempi di generazione su dataset di grandi dimensioni.
4. **Monitoraggio e statistiche:** una barra di avanzamento `tqdm` mostra in tempo reale lo stato della generazione. Al termine, il sistema stampa statistiche aggregate: numero totale di immagini prodotte, tempo totale, immagini per secondo.

Ogni task viene eseguito dalla funzione `run_single_file()` definita in `gen_ecg_image_from_data.py`, che costituisce il nucleo dell'intera pipeline di generazione per un singolo record.

2.4 Segmentazione del Segnale e Gestione dei Layout

La funzione `get_paper_ecg()` in `extract_leads.py` gestisce la conversione del segnale in frame pronti per il rendering. Il segnale ECG, tipicamente della durata di 10 secondi, viene suddiviso in finestre temporali la cui estensione dipende dal layout scelto.

2.4.1 Layout Supportati

Il sistema supporta quattro configurazioni di stampa, che replicano i formati utilizzati nella pratica clinica:

- **12x1:** tutte e 12 le derivazioni disposte in una singola colonna. Ogni derivazione mostra 10 secondi di segnale.
- **6x2:** le 12 derivazioni divise in due colonne da 6. Le derivazioni standard (I–aVF) occupano la prima colonna, quelle precordiali (V1–V6) la seconda. Ciascuna colonna mostra 5 secondi di segnale.
- **3x4_1 (formato clinico standard):** le 12 derivazioni disposte in 3 righe da 4 colonne, ciascuna mostrante 2.5 secondi di segnale da un segmento temporale diverso. Una derivazione ritmo (tipicamente la II) viene aggiunta come striscia completa da 10 secondi in calce all'immagine (*full mode*).
- **6x1_split:** le derivazioni vengono distribuite su due pagine separate, con le prime 6 nella prima pagina e le restanti 6 nella seconda.

2.4.2 Segmentazione Temporale per il Formato 3x4

Il formato clinico standard (3x4) merita una descrizione più dettagliata, poiché il campionamento dei segmenti non è contiguo ma *sfasato*: la prima colonna (derivazioni I, II, III) mostra i campioni da 0 a 2.5s, la seconda colonna (aVR, aVL, aVF) mostra i campioni da 2.5s a 5s, la terza (V1–V3) da 5s a 7.5s, e la

quarta (V4–V6) da 7.5s a 10s. La derivazione ritmo in fondo alla pagina mostra invece l'intera finestra da 10 secondi. In questo modo, su carta si rappresenta l'attività elettrica completa del cuore in un'unica pagina, con le colonne allineate temporalmente.

Questo schema di segmentazione è implementato in `extract_leads.py` tramite un calcolo dello `shiftedStart` per ogni derivazione in funzione della sua appartenenza alla colonna corrispondente nella struttura `format_4_by_3` definita in `config.yaml`.

Il parametro `-mask_unplotted_samples` consente di mascherare con NaN le porzioni di segnale non rappresentate graficamente nel file WFDB di output, permettendo un allineamento preciso tra immagine e segnale digitale.

2.5 Il Motore di Rendering: `ecg_plot.py`

Il cuore del sistema è la funzione `ecg_plot()` in `ecg_plot.py`. Essa riceve il dizionario del segnale segmentato e produce un'immagine PNG rispettando le porzioni fisiche di un ECG stampato su carta standard.

2.5.1 Calibrazione Fisica della Griglia

Uno dei requisiti fondamentali per l'addestramento di modelli di digitizzazione è che la corrispondenza tra pixel e grandezze fisiche sia nota e precisa. Il sistema utilizza come riferimento le convenzioni cliniche standard:

- **Asse temporale:** una quadretto grande corrisponde a 0.2 secondi; ogni suddivisione minore a 0.04 secondi.
- **Asse di ampiezza:** una quadretto grande corrisponde a 0.5 mV.
- **Dimensioni della griglia:** i valori in pollici per ogni cella sono `y_grid_inch = x_grid_inch = 5/25.4` (5 mm convertiti in pollici), garantendo la proporzione 1:1 tra gli assi.

Il canvas è configurato con dimensioni fisiche fisse (default: 11×8.5 pollici, corrispondente al formato *letter*), e la risoluzione in DPI determina il numero effettivo di pixel dell'immagine. A 200 DPI si ottiene un'immagine di circa 2200×1700 pixel, mentre il parametro `-random_resolution` consente di campionare casualmente la risoluzione nell'intervallo $[50, r]$ per aumentare la varietà del dataset.

La relazione tra risoluzione D (in DPI) e frequenza di campionamento equivalente dell'immagine f'_s è [?]:

$$f'_s = \frac{D}{1.016} \quad [\text{Hz}]$$

Questo implica che risoluzioni inferiori a 150 DPI producono una frequenza di campionamento equivalente insufficiente per rappresentare fedelmente i transienti ad alta frequenza del segnale ECG.

2.5.2 Configurazione del Canvas Matplotlib

Il rendering è eseguito interamente tramite la libreria `Matplotlib` con backend `Agg` (non interattivo), che consente la generazione di immagini in processi paralleli senza dipendenze da un display fisico. La figura è inizializzata con:

```
fig, ax = plt.subplots(figsize=(width, height), dpi=resolution)
fig.subplots_adjust(hspace=0, wspace=0, left=0, right=1, bottom=0, top=1)
```

I margini vengono azzerati in modo che l'area di disegno occupi esattamente l'intera dimensione fisica specificata, senza bordi aggiuntivi introdotti da `Matplotlib`. Il padding bianco attorno all'immagine, quando richiesto, viene aggiunto *a posteriori* tramite la libreria `Pillow`, estendendo il canvas con pixel bianchi per il numero di pollici specificato da `-pad_inches`.

2.5.3 Posizionamento delle Derivazioni

Per ogni derivazione nel vettore `lead_index`, la funzione calcola le coordinate di rendering in unità del sistema di riferimento del grafico (mV e secondi):

- **Posizione verticale (`y_offset`):** calcolata come `initial_y_offset + (row_idx * row_height)`, dove `row_height` è determinato in modo tale che tutte le derivazioni si distribuiscano uniformemente sull'altezza del canvas, tenendo conto del numero di righe e di un fattore di spaziatura verticale (`v_space_factor`).
- **Posizione orizzontale (`x_offset`):** calcolata come `current_col * (secs + col_gap)`, dove `col_gap` è un parametro aggiunto per inserire un gap deterministico minimo tra le colonne.

Il segnale di ogni derivazione viene tracciato come una polilinea:

```
x_vals = np.arange(0, len(ecg[leadName])*step, step) + x_offset + x_gap
y_vals = ecg[leadName] + y_offset
ax.plot(x_vals, y_vals, linewidth=line_width, color=color_line)
```

2.5.4 Colori della Griglia e Stile

Il sistema supporta tre modalità di colorazione della griglia:

1. **Scala di grigi (BW):** tutti gli elementi disegnati in nero, attivato dal parametro `-random_bw`.
2. **Colori standard:** cinque palette predefinite che replicano i colori tipici degli ECG clinici (marrone, rosa, blu, verde, rosso), selezionabili tramite `-standard_grid_color`.
3. **Colori casuali:** le componenti RGB dei colori della griglia vengono campionate da distribuzioni uniformi, garantendo un'ampia varietà cromatica nel dataset. Attivato da `-random_grid_color`.

Le linee della griglia seguono le convenzioni cliniche: linee maggiori (passo 0.2s / 0.5mV) con spessore e saturazione maggiore, linee minori (passo 0.04s / 0.1mV) più sottili e chiare. Un ulteriore parametro `-prob_dotted` consente di sostituire le linee minori con una griglia di punti, simulando alcuni formati di stampa utilizzati in dispositivi ECG più datati.

2.5.5 Impulso di Calibrazione

L'impulso di calibrazione (o *DC pulse*) è un'onda quadra di ampiezza 1 mV e durata 0.2 secondi posizionata a sinistra di ogni derivazione. Esso è presente negli ECG clinici come riferimento per la taratura dell'ampiezza. Il sistema supporta due modalità:

- **Single:** un unico impulso posizionato casualmente su una riga, con possibilità di clipping parziale (parametri `-prob_pulse_clip` e `-pulse_clip_fraction`).
- **Per lead:** un impulso per ogni derivazione, precedente al segnale.

La probabilità che l'impulso sia presente in un'immagine è controllata da `-calibration_pulse`. L'ampiezza e la durata dell'impulso sono parametrizzabili tramite `-dc_pulse_amplitude` e `-dc_pulse_duration`.

2.5.6 Augmentation Geometrica in Fase di Rendering

Un contributo originale apportato al sistema riguarda l'introduzione di perturbazioni geometriche direttamente nella fase di rendering, prima della rasterizzazione. Il parametro `-prob_shift` controlla la probabilità che ciascun elemento visivo (derivazione, etichetta, impulso di calibrazione, colonna) venga traslato di una quantità casuale:

- Traslazione verticale delle derivazioni: fino a `-max_shift_y` mV.
- Traslazione orizzontale delle colonne: fino a `-max_shift_col_x` secondi.
- Traslazione dell'impulso di calibrazione: fino a `-max_shift_pulse`.
- Traslazione dell'etichetta della derivazione: fino a `-max_shift_name`.

Questi spostamenti simulano le irregolarità di allineamento tipiche dei sistemi di stampa analogici e delle scansioni inclinate, aumentando la robustezza dei modelli addestrati su questo dato.

2.5.7 Annotazioni e Bounding Box

Per ogni derivazione, il sistema traccia la corrispondenza tra pixel dell'immagine e campioni del segnale, memorizzando le coordinate pixel di ogni punto plottato nel dizionario `json_dict`. Sono inoltre disponibili le *bounding box* rettangolari che delimitano sia il tracciato del segnale (`-lead_bbox`) che l'etichetta testuale (`-lead_name_bbox`). Queste annotazioni vengono serializzate in un file JSON per ogni immagine generata, rendendo il dataset utilizzabile direttamente per task di object detection e di digitizzazione supervisionata.

2.5.8 Generazione dell'Immagine Pulita

Il flag `-generate_clean`, introdotto come estensione originale, abilita la generazione contestuale di una versione pulita di ogni immagine: sfondo bianco, nessuna griglia, nessun artefatto, solo i tracciati delle derivazioni in nero. Questa immagine costituisce la *ground truth* visiva per task di segmentazione semantica, consentendo ai modelli di apprendere la separazione tra segnale e disturbi.

2.6 Output del Sistema e Configurazione JSON

Al termine della generazione di ogni immagine, il sistema produce i seguenti file:

- **Immagine PNG sintetica:** il file immagine principale, con griglia, derivazioni, etichette e tutti gli artefatti richiesti.
- **Immagine PNG pulita (opzionale):** la ground truth visiva, priva di griglia e disturbi.
- **File WFDB aggiornato:** una copia dei file `.hea` e `.dat` che riflette la segmentazione temporale applicata, con i campioni non visualizzati eventualmente mascherati con NaN.
- **File JSON di configurazione (opzionale):** serializza i parametri utilizzati per generare l'immagine (risoluzione, DPI della griglia, presenza del pulse di calibrazione, colori, flag di augmentation), nonché le coordinate delle bounding box e i pixel plottati per ogni derivazione. Il livello di dettaglio è controllato dal parametro `-store_config` (livello 1: attributi di alto livello; livello 2: informazioni complete).

2.7 Sistema di Configurazione

La configurazione del sistema è stratificata su due livelli:

- **File YAML (`config.yaml`):** contiene i parametri statici della griglia (durata della carta, passo della griglia, layout delle derivazioni per i diversi formati). Questi valori, originariamente fissi nel codice, sono stati estratti in questo file di configurazione come contributo originale al progetto, rendendo il sistema più facilmente adattabile a standard clinici diversi da quello di default.
- **Argomenti da riga di comando:** espongono oltre 50 parametri che controllano ogni aspetto della generazione, dal layout alla risoluzione, dai colori alle probabilità di applicazione di ciascun artefatto. I parametri con prefisso `-deterministic_*` consentono di fissare il valore di un disturbo invece di campionarlo casualmente, utile per esperimenti ripetibili o per la generazione di dataset bilanciati.

Questa architettura a due livelli separa nettamente i parametri strutturali del formato ECG (invarianti) dai parametri operativi della generazione (variabili per ogni run), consentendo di lanciare esperimenti con diverse configurazioni senza modificare il codice sorgente.

Nel prossimo capitolo verrà analizzata nel dettaglio la pipeline di artefatti, descrivendo i meccanismi con cui testo scritto a mano, pieghe e rugosità della carta e distorsioni fotometriche vengono iniettati sull'immagine sintetica per simulare le condizioni reali dei documenti clinici.

Bibliography

- [1] Yao Li, Qixun Qu, Meng Wang, Liheng Yu, Jun Wang, Linghao Shen, and Kunlun He. Deep learning for digitizing highly noisy paper-based ecg records. *Computers in Biology and Medicine*, 127:104077, 2020.

There will be the thanks to precious people here.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.